

MẠNG NƠ-RON HỒI QUY (RNN)

Từ Hạn Chế của MLP & 1D-CNN đến Nguyên Lý Xử Lý Dữ Liệu Chuỗi có Trạng Thái Ẩn

Nhóm thực hiện:

Văn Thị Mai Linh Nguyễn Văn Minh Lực Giáp Minh Hiếu Nguyễn Nam Hải

Chuyên đề Học Sâu (Deep Learning)

*"Feedforward nhìn thế giới qua từng bức ảnh rời rạc;
RNN thấu hiểu dữ liệu qua dòng chảy liên tục của thời gian."*

Bản chất Dữ liệu dạng Chuỗi (Sequential Data)

Thứ tự xuất hiện quyết định hoàn toàn ngữ nghĩa của dữ liệu

- **Đặc tính trật tự:** Mỗi điểm dữ liệu tại thời điểm t luôn mang mối tương quan mật thiết với các điểm quá khứ $t - 1, t - 2, \dots$
- **Hai dạng chính:** Dữ liệu token rời rạc (Text, DNA) và dữ liệu chuỗi số liên tục (Giá vàng, cổ phiếu, nhịp tim).

Chuỗi Rời rạc (NLP / Văn bản)

”Hà Nội không mưa” $\neq$ ”Mưa không Hà Nội”

→ Câu đầu có nghĩa rõ ràng, câu sau vô nghĩa hoặc biến đổi ngữ nghĩa.

Chuỗi Liên tục (Time-Series)

Giá vàng: $[82.5 \rightarrow 83.1 \rightarrow 84.0 \rightarrow 85.2]$ (Tăng)

Giá vàng: $[85.2 \rightarrow 84.0 \rightarrow 83.1 \rightarrow 82.5]$ (Giảm)

Bài toán Dự đoán Phần tử Kế tiếp (Next Step Prediction)

Mục tiêu: Cho chuỗi quan sát quá khứ $X = (x_1, x_2, \dots, x_t)$, dự đoán giá trị tương lai x_{t+1} .

Bài toán 1: Next Word Prediction (NLP)

Ngữ cảnh: "We" $\rightarrow$ "are" $\rightarrow$ "learning" $\rightarrow$ "AI" $\rightarrow$ [?]
 $\rightarrow$ Target kỳ vọng: "is"

Bài toán 2: Dự báo Chuỗi thời gian (Time-Series)

Cửa sổ trượt (30 ngày lịch sử):
 $[P(t-29), P(t-28), \dots, P(t-1), P(t)] \rightarrow [P(t+1)]$
 $\rightarrow$ Dự báo biến động ngày 31.

Hai Pipeline Tiền xử lý Đặc trưng

Dữ liệu thô không thể đưa trực tiếp vào mạng mà phải đi qua pipeline chuẩn hóa:

Văn bản (Text Pipeline)

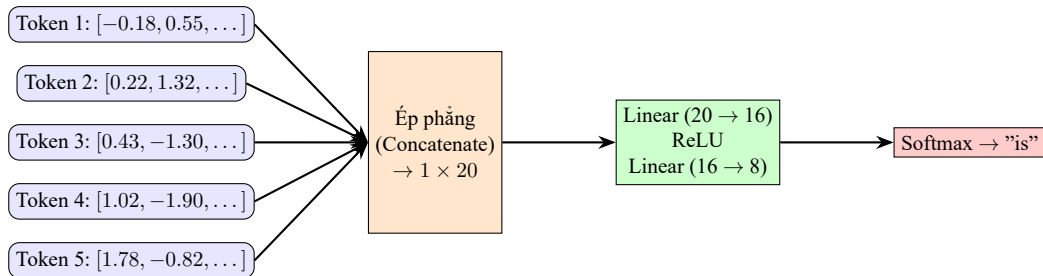
- ❶ Văn bản thô: "giá vàng tăng"
- ❷ Tokenizer: ["giá", "vàng", "tăng"]
- ❸ Vocab Index: [2, 3, 4]
- ❹ Padding: [2, 3, 4, 0, 0]
- ❺ Embedding: Tensor [Batch, Seq_Len, d]

Chuỗi số (Time-Series Pipeline)

- ❶ File CSV thô: Date, Price
- ❷ Làm sạch & Chia tập: Train 70%, Val 15%, Test 15%
- ❸ Scaling: $\log_{1p}(\text{Price}) \rightarrow \text{MinMax}$ (chỉ fit trên Train)
- ❹ Windowing: Cửa sổ trượt 30 ngày
- ❺ Đầu vào: Tensor [N, 30, 1]

Tiếp cận bằng MLP & Cơ chế Ghép phẳng (Flatten)

Mạng truyền thẳng (MLP) bắt buộc phải biến toàn bộ ngữ cảnh thành một vector phẳng duy nhất.



Ba Điểm nghẽn Bản chất của MLP

1. Cửa sổ cố định (Fixed Window)

Ma trận W gắn cứng với kích thước cố định L .

- Câu ngắn: Bắt buộc nhồi padding dư thừa.
- Câu dài hơn L : Cắt cụt, mất hoàn toàn dữ liệu.

2. Mất tính tuần tự (Non-sequential)

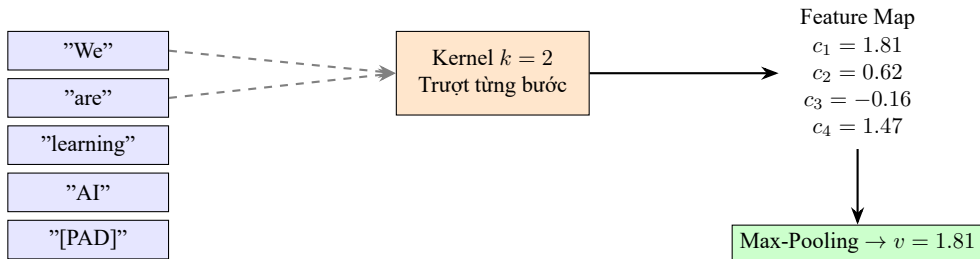
Ép phẳng làm mất trục thời gian. Mạng xem từ ở vị trí 1 và từ ở vị trí 5 là hai tọa độ độc lập, không nhận diện được quy luật dịch chuyển.

3. Bùng nổ tham số (Parameter Explosion)

Số tham số tầng Linear = $W_1 \times (L \times d)$. Độ dài ngữ cảnh tăng gấp đôi khiến trọng số tăng gấp đôi, dễ gây quá khớp (overfitting) và tốn bộ nhớ.

Bước tiến 1D-CNN: Cửa sổ Trượt Kernel (N-gram)

1D-CNN khắc phục việc gán cứng vị trí bằng cách dùng Kernel trượt qua từng bước thời gian:



Vì sao 1D-CNN vẫn chưa tối ưu cho chuỗi dài?

Vấn đề 1: Tầm nhìn bị bó hẹp (Limited Receptive Field)

Kernel ($k = 2$) chỉ bắt được từng cặp từ đứng cạnh nhau (Bigram). Không thể liên kết chủ ngữ ở đầu câu với vị ngữ ở cuối một đoạn văn dài.

Vấn đề 2: Max-Pooling xóa bỏ thứ tự thời gian

- c_1 ("We are") = 1.81
- c_2 ("are learning") = 0.62
- c_3 ("learning AI") = -0.16
- c_4 ("AI [PAD]") = 1.47

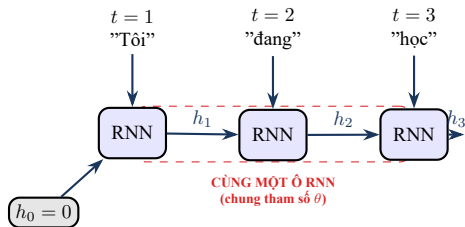
Max-Pooling lấy giá trị lớn nhất là 1.81, nhưng **hoàn toàn quên** 1.81 xuất hiện ở đầu hay cuối câu!

Bảng So sánh: MLP vs. 1D-CNN vs. RNN

Tiêu chí đối sánh	MLP (Feedforward)	1D-CNN (Convolutional)	Recurrent Neural Network (RNN)
Độ dài ngữ cảnh	Cố định cứng, phải đệm padding	Cửa sổ trượt cố định theo k	Co giãn linh hoạt theo độ dài chuỗi
Bảo toàn thứ tự	Bị xóa sạch sau phép Flatten	Giữ được trật tự cục bộ N-gram	Duy trì trật tự toàn vẹn theo thời gian
Chia sẻ trọng số	Không chia sẻ (Độc lập vị trí)	Chia sẻ trọng số theo không gian	Chia sẻ trọng số xuyên suốt thời gian
Bộ nhớ tích lũy	Không có bộ nhớ	Gián tiếp qua độ sâu filter	Bộ nhớ trạng thái ẩn (h_t) trực tiếp
Tăng trưởng tham số	Tăng vọt theo độ dài ($L \cdot d$)	Phụ thuộc số filter, độc lập L	Hằng số, hoàn toàn không phụ thuộc L

Ý tưởng Cốt lõi của RNN: Trạng thái Ẩn (h_t)

Thực tế: Con người không bắt đầu tư duy lại từ đầu ở từng chữ, mà liên tục duy trì một dòng ý thức **nén dần thông tin** quá khứ.



h_t = "trí nhớ" nén toàn bộ quá khứ:

- h_1 : chỉ nhớ "Tôi"
- h_2 : nhớ "Tôi" + "đang"
- h_3 : nhớ trọn câu "Tôi đang học"

Phương trình đệ quy của RNN

Mỗi bước chỉ cần **trạng thái liền trước** h_{t-1} và **đầu vào hiện tại** x_t :

$$h_t = f_{\theta}(h_{t-1}, x_t)$$

- **Tóm tắt lịch sử vô hạn:** h_{t-1} đã chứa cả chuỗi quá khứ $x_1, \dots, x_{t-1}$, nên quá khứ dài bao nhiêu cũng được nén vào vector **cố định** d_h chiều.
- **Cập nhật từng bước $O(1)$:** mỗi token đến chỉ cần **một** phép cập nhật; xử lý kiểu streaming, không phải lưu toàn chuỗi trong bộ nhớ.
- **Khác MLP của số trượt:** MLP phải thấy đủ L phần tử trong cửa sổ; RNN duy trì "tóm tắt động" h_t **không phụ thuộc độ dài** chuỗi.

Công thức Toán học – Quá trình Forward

Ba phương trình forward tại bước t , với $x_t \in \mathbb{R}^{d_x}$ (đầu vào), $h_t \in \mathbb{R}^{d_h}$ (trạng thái ẩn), $\hat{y}_t \in \mathbb{R}^{d_y}$ (dự đoán):

$$a_t = \mathbf{W}x_t + \mathbf{U}h_{t-1} + b_h$$

$$h_t = \tanh(a_t)$$

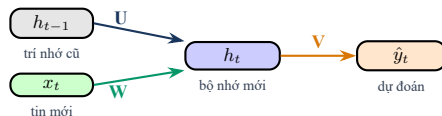
$$\hat{y}_t = \mathbf{V}h_t + b_y$$

tổng nhập: ghép tin mới với trí nhớ cũ

trạng thái ẩn mới, truyền sang bước $t + 1$

đọc bộ nhớ h_t để đưa ra dự đoán

Tham số	Kích thước	Kết nối	Vai trò
$\mathbf{W}$	$d_h \times d_x$	Vào $\rightarrow$ Ẩn	Đọc tin mới x_t
$\mathbf{U}$	$d_h \times d_h$	Ẩn $\rightarrow$ Ẩn	Giữ trí nhớ h_{t-1}
$\mathbf{V}$	$d_y \times d_h$	Ẩn $\rightarrow$ Ra	Đọc ra kết quả
b_h, b_y	d_h, d_y	Bias	Dịch ngưỡng



- **Vì sao hàm tanh?** Tâm 0, chặn $(-1, 1)$ nên chuẩn của h_t ổn định qua các bước, đạo hàm $1 - \tanh^2(a_t)$ gọn; *sigmoid* lệch dương (trung bình 1/2), đạo hàm ≤ 0.25 làm gradient tắt nhanh; *ReLU* không chặn trên, trạng thái cộng dồn dễ bùng nổ.
- **Tên PyTorch tương ứng:** $\mathbf{W}$ = weight_ih_10, $\mathbf{U}$ = weight_hh_10 (Keras: kernel, recurrent_kernel); mất mát tổng theo thời gian $L = \sum_{t=1}^T \ell(\hat{y}_t, y_t)$ (MSE cho hồi quy).

Ví dụ Tính Toán Forward Từng Bước (số cụ thể)

$$d_x = 2, \ d_h = 2, \ d_y = 1; \quad W = \begin{pmatrix} 0.5 & 0.2 \\ -0.1 & 0.4 \end{pmatrix}, \ U = \begin{pmatrix} 0.3 & 0.0 \\ 0.2 & 0.5 \end{pmatrix}, \ V = \begin{pmatrix} 1.0 & -1.0 \end{pmatrix},$$
$$\mathbf{x}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \ \mathbf{h}_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \ \mathbf{b}_h = \mathbf{0}, \ b_y = 0.1$$

Bước	Phép tính	Ý nghĩa	Kết quả ($t = 1$)
1	$W \cdot \mathbf{x}_1$	Đọc tin mới	(+0.500, -0.100)
2	$U \cdot \mathbf{h}_0$	Chưa có ký ức (trang trắng)	(+0.000, +0.000)
3	$\mathbf{h}_1 = \tanh(W \cdot \mathbf{x}_1 + U \cdot \mathbf{h}_0 + \mathbf{b}_h)$	Ký ức mới (vector)	(+0.462, -0.100)
4	$\hat{y}_1 = V \cdot \mathbf{h}_1 + b_y$	Đầu ra (số)	+0.662

Bước $t = 2$: lặp nguyên hệ thống. Thay $\mathbf{x}_2 = (0, 1)$, lấy $\mathbf{h}_1$ vừa tính làm *trí nhớ cũ* (lúc này $U \cdot \mathbf{h}_1 \neq \mathbf{0}$: thành phần hồi quy xuất hiện lần đầu); bộ trọng số (W, U, V) giữ nguyên tuyệt đối, chạy dần đến $t = T$.

Shape flow: $\mathbf{x} \in \mathbb{R}^{T \times d_x} \longrightarrow \mathbf{h} \in \mathbb{R}^{T \times d_h} \longrightarrow \hat{\mathbf{y}} \in \mathbb{R}^{T \times d_y}$

Ghi chú: PyTorch `nn.RNN(2, 2) + nn.Linear(2, 1)` nạp đúng W, U, V , đặt 2 bias của `nn.RNN` về 0 và bias `Linear` = 0.1 thì chạy đúng ví dụ (sai số $< 10^{-6}$).

Nguyên lý Chia sẻ Trọng số (Weight Sharing)

Chuỗi dài 5 bước hay 5.000 bước, RNN đều **dùng chung một bộ ma trận** $\mathbf{W}, \mathbf{U}, \mathbf{V}$ duy nhất:

$$\text{Thời điểm } t = 1: \quad h_1 = \tanh(\mathbf{W} \cdot x_1 + \mathbf{U} \cdot h_0 + b_h) \rightarrow y_1 = \mathbf{V} \cdot h_1$$

$$\text{Thời điểm } t = 2: \quad h_2 = \tanh(\mathbf{W} \cdot x_2 + \mathbf{U} \cdot h_1 + b_h) \rightarrow y_2 = \mathbf{V} \cdot h_2$$

$\vdots$ áp dụng cùng một bộ $\mathbf{W}, \mathbf{U}, \mathbf{V}$ tới $t = T$

Tổng số tham số P — không chứa T

$$P = \underbrace{d_h \cdot d_x}_{\mathbf{W}} + \underbrace{d_h^2}_{\mathbf{U}} + \underbrace{d_y \cdot d_h}_{\mathbf{V}} + \underbrace{d_h}_{b_h} + \underbrace{d_y}_{b_y}$$

Ví dụ $d_x = 2, d_h = 64, d_y = 1$:

$P = 128 + 4096 + 64 + 64 + 1 = 4353$ tham số cho **mọi độ dài chuỗi**.

So với MLP cửa sổ $L = 30$

- **MLP** ($d_x = 2, 64$ nơ-ron ẩn): trọng số tầng đầu $= 30 \cdot 2 \cdot 64 = 3840$ (+64 bias), chỉ dùng cho **một độ rộng cửa sổ 30**.
- **RNN**: $\mathbf{W} = 2 \cdot 64 = 128$ tham số nhưng **tái sử dụng 30 lần**.

Bốn lợi ích cốt lõi

- **Tham số hằng số theo T** — chuỗi dài bao nhiêu cũng không tăng thêm tham số.
- **Tổng quát hóa**: mẫu hình xuất hiện ở đầu hay cuối chuỗi đều được nhận diện giống nhau.
- **Tự điều chuẩn hóa**: ít tham số hơn trên cùng dữ liệu $\rightarrow$ kháng quá khớp tốt hơn.
- **Analogy CNN**: kernel CNN chia sẻ trọng số theo **KHÔNG GIAN**, RNN chia sẻ theo **THỜI GIAN**.

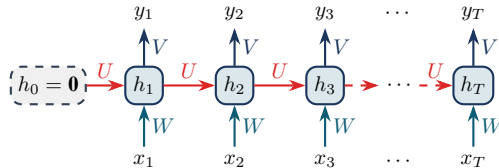
Đồ thị Tính toán dạng Mở cuộn (Unfolded Computation Graph)

Một ô RNN hồi quy khi mở cuộn trở thành một **mạng truyền thẳng cực sâu** có độ sâu đúng bằng T :

DẠNG CUỘN (Folded)



DẠNG MỞ CUỘN (Unfolded)



CÙNG MỘT Ô

— dùng chung W , U , V mọi thời điểm

Mở cuộn = Mạng sâu T tầng

Truyền thẳng **cực sâu**, độ sâu đúng bằng $T \rightarrow$ **BPTT** chính là backprop chạy trên đúng đồ thị này.

U = Đường trí nhớ

Cạnh ngang $h_{t-1} \rightarrow h_t$ (màu đỏ) đưa quá khứ vào hiện tại; **cắt nó** thì mất hết thông tin quá khứ.

T giãn mãi — tham số cố định

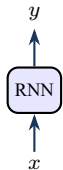
Cùng bộ W, U, V tái sử dụng $\rightarrow$ đồ thị dài thêm bao nhiêu cũng **không thêm trọng số** nào.

Các Dạng Kiến trúc Tương tác Chuỗi (Phần 1)

Tùy số lượng bước vào / bước ra, một ô RNN tạo thành các kiến trúc tương tác chuỗi khác nhau:

One-to-One

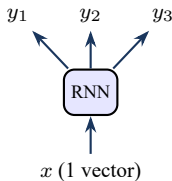
1 bước vào $\rightarrow$ 1 bước ra



Phân loại 1 ảnh — mốc đối chiếu, chưa khai thác ngữ cảnh chuỗi

One-to-Many

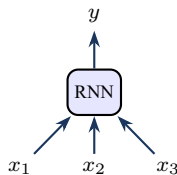
1 bước vào $\rightarrow$ chuỗi ra



Sinh chú thích ảnh (image captioning), sinh nhạc

Many-to-One

chuỗi vào $\rightarrow$ 1 đầu ra

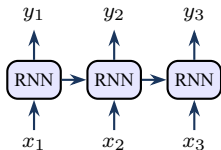


Phân loại cảm xúc review; dự báo giá ngày mai — dạng dùng trong phần thực nghiệm của bài

Các Dạng Kiến trúc Tương tác Chuỗi (Phần 2)

Many-to-Many (Đồng bộ)

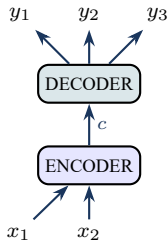
mỗi bước vào $\rightarrow$ 1 nhãn ra



Gán nhãn từ loại POS / NER: mỗi token ra đúng 1 nhãn

Seq2Seq (Encoder-Decoder)

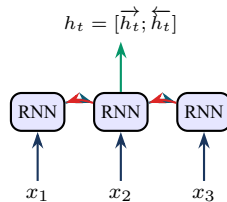
độ dài ra $\neq$ độ dài vào



Encoder nén chuỗi vào thành vector ngữ cảnh c ; Decoder sinh chuỗi ra: dịch máy, tóm tắt

Bidirectional RNN

nhìn cả 2 chiều thời gian



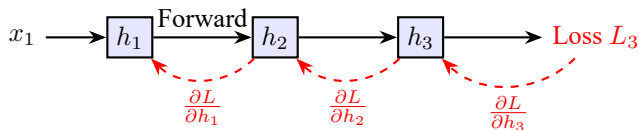
NER, embedding; không dùng dự báo tương lai — nhìn được tương lai = rò rỉ dữ liệu

Liên hệ bài toán asm6

Chuỗi giá quá khứ $\rightarrow$ 1 giá ngày mai $\Rightarrow$ bài toán thực nghiệm thuộc dạng **Many-to-One**.

Cơ chế Huấn luyện BPTT (Backpropagation Through Time)

Mô hình tính toán sai số tại mọi thời điểm và lan truyền ngược gradient ngược dòng thời gian từ T về 1:



Công thức Gradient cho U :

$$\frac{\partial L}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial L_t}{\partial h_t} \cdot \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_k}{\partial U}$$

Vấn đề Triệt tiêu Gradient (Vanishing Gradient)

Bản chất: Phép nhân chuỗi ma trận U liên tiếp qua $T - k$ bước thời gian:

$$\frac{\partial L}{\partial h_k} = \frac{\partial L}{\partial h_t} \times [\text{diag}(1 - h^2) \cdot U] \times \cdots \times [\text{diag}(1 - h^2) \cdot U]$$

(nhân $T - k$ lần, đạo hàm $\tanh(z)$ nằm trong $(0, 1]$)

Hậu quả khi Trị riêng lớn nhất của $U < 1$

$$\|U^{T-k}\| \longrightarrow 0 \quad (\text{khi chuỗi dài ra})$$

Tín hiệu đạo hàm ở các bước đầu tiên suy giảm theo hàm mũ về 0. Trọng số không thể cập nhật để học các liên kết ngữ cảnh quá khứ xa.

Hiện tượng Nổ Gradient & Gradient Clipping

Hiện tượng Nổ Gradient (Exploding Gradient)

- Khi trị riêng lớn nhất của $U > 1$: $\|U^{T-k}\|$ bùng nổ vô tận theo hàm mũ.
- Bước nhảy trọng số quá lớn phá hủy vùng cực tiểu, làm tràn số (NaN / Inf).

Giải pháp: Cắt tỉa Gradient (Gradient Norm Clipping)

$$g_{\text{clipped}} = g \cdot \frac{\theta}{\|g\|} \quad \text{khi } \|g\| > \theta$$

Vector ban đầu quá dài sẽ bị co ngắn lại cho bằng θ , nhưng vẫn giữ nguyên hướng (hướng Gradient).

Tổng kết Điểm nghẽn của Simple RNN

1. Không có Cơ chế Chọn lọc (Unselective Memory)

Mọi bước thời gian đều bị nén qua một hàm tanh duy nhất, thông tin quan trọng dễ bị thông tin rác làm loãng.

2. Mất Trí nhớ Dài hạn (Short-term Memory Horizon)

Do triệt tiêu gradient, RNN thực tế chỉ nhớ được từ 5 đến 10 bước thời gian gần nhất.

3. Bắt buộc Tính toán Tuần tự (Sequential Bottleneck)

h_t chỉ tính được khi đã có $h_{t-1} \rightarrow$ KHÔNG THỂ SONG SONG HÓA trên GPU như Transformer hay CNN.

Thiết lập Thực nghiệm Chuỗi Tài chính

Kiểm chứng hiệu năng của RNN trên 2 tập dữ liệu thực tế lớn từ Kaggle:

- **Tập 1:** Chuỗi giá vàng & bạc hàng ngày (Gold & Silver daily prices, 1968–2021).
- **Tập 2:** Chuỗi biến động giá & khối lượng cổ phiếu Amazon (AMZN, 1997–2023).

Cấu hình Thực nghiệm

- **Cửa sổ trượt:** 30 ngày lịch sử ($L = 30$) để dự đoán bước tiếp theo ($t + 1$).
- **Baseline đối chứng:** Phương pháp Naive / Persistence ($\hat{y}_{t+1} = y_t$ — Lấy giá hôm nay đoán cho ngày mai).

Kết quả Thực nghiệm & Hiện tượng Nghịch lý

Bảng so sánh độ lệch sai số căn quân phương (RMSE trên tập Test):

Tập dữ liệu	Naive Baseline	PyTorch RNN	Keras RNN	Đánh giá
Giá Vàng (Gold)	15.48	15.45	16.42	<i>Keras RNN chạy tệ hơn cả Naive</i>
Cổ phiếu AMZN	3.14	3.13	3.18	<i>Xấp xỉ phương pháp Naive</i>

Nghịch lý Quan sát

Một mạng nơ-ron học sâu phức tạp huấn luyện hàng chục epoch kết quả lại chỉ tương đương, thậm chí kém hơn cả một thuật toán "lười biếng" lấy giá hôm trước làm dự đoán!

Bản chất Thất bại: Dịch chuyển Phân phối (Distribution Shift)

Tại sao Simple RNN lại thất bại ở bài toán dự báo giá kim loại?

Phân phối Dữ liệu Train vs Test

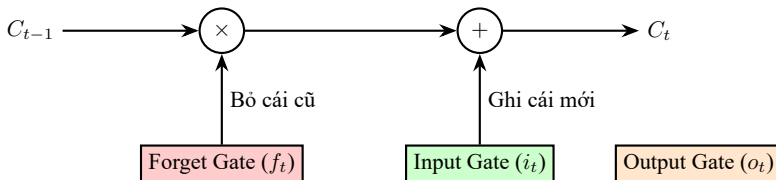
- **Tập Train:** Giá cao nhất = 850.00 USD (ĐÃ ĐƯỢC HỌC - Nội suy tốt)
- **Tập Test:** Giá thấp nhất = 1,049.40 USD (CHƯA GẶP BAO GIỜ - Ngoại suy bất lực)
- $\rightarrow 100$

Bài học & Giải pháp khắc phục

- Mạng nơ-ron không thể ngoại suy trên vùng số liệu mới.
- **Giải pháp:** Không dự đoán mức giá tuyệt đối, phải chuyển thành tỷ suất sinh lời dừng (**Log-return**):
$$r_t = \ln(P_t/P_{t-1}).$$

Bước Tiến hóa LSTM: Đường dẫn Trí nhớ Dài hạn

Long Short-Term Memory bổ sung đường truyền thẳng **Cell State** (C_t) không bị cản trở bởi phép nhân ma trận lặp lại:



- **Cổng Quên (f_t):** Xóa bỏ các ký ức không còn phù hợp.
- **Cổng Ghi (i_t):** Chọn lọc thông tin mới đáng chú ý nạp vào.
- **Cổng Xuất (o_t):** Quyết định trích xuất thông tin gì ra ngoài trạng thái ẩn.

Tinh giản với GRU (Gated Recurrent Unit)

GRU rút gọn cấu trúc của LSTM, gộp Cell State và Hidden State thành một trạng thái duy nhất:

Cổng Cập nhật (Update Gate z_t)

Cân bằng giữa việc giữ lại trí nhớ cũ và ghi đè trạng thái mới (gộp vai trò của Forget và Input gate).

Cổng Đặt lại (Reset Gate r_t)

Quyết định xem trạng thái quá khứ sẽ kết hợp bao nhiêu với đầu vào hiện tại.

Tối ưu hiệu suất

Giảm bớt 25

TIẾN TRÌNH CÔNG NGHỆ MÔ HÌNH HÓA DỮ LIỆU CHUỖI

MLP / 1D-CNN (2012)	RNN THUẦN (1986-1990)	LSTM / GRU (1997-2014)	TRANSFORMER (2017+)
• Ép phẳng, trượt cục bộ • Mất tính tuần tự dài • Chỉ phù hợp chuỗi ngắn	• Bộ nhớ ẩn h_t • Bị triệt tiêu gradient • Dễ nổ đạo hàm	• Cơ chế cổng kiểm soát • Nhớ dài hạn rất tốt • Chuẩn mực Time-series	• Cơ chế Self-Attention • Xử lý song song tuyệt đối • Thống trị lĩnh vực LLM

Thông điệp kết luận

Simple RNN đã đặt nền móng lý thuyết mang tính lịch sử về việc chia sẻ trọng số thời gian và trạng thái ẩn. Hiểu rõ các điểm nghẽn toán học của RNN chính là chìa khóa để làm chủ toàn bộ các mô hình học sâu chuỗi hiện đại.